1. Have minimal experience with all these protocols. So did not mentioned any of them in the code.
2. Best way to save power depends on selected mcu. But from my experience, there are usually 4 signifacant working modes:
    a. Mcu fully active. All clocks and all periphery operational.
    b. Limited mode (in cod it is mentioned as “halt”). Usually mcu clock is turned off, but periphery can work.
    c. Stop mode. Mcu clock stopped. Works just those peripheries that are clocked from external crystal resonator. After irq device wakes up without full reinitialization.
    d. Standby mode. After irq device do full initialization.

There is no guarantee that certain mcu has all of them. Since there was no indication what mcu to use, code was developed with three modes in mind: fully active, limited and stop mode.

Two doubly linked lists need to be created to store periphery power modes (this part unfinished in code):

- One to store list of peripheries that needs mcu to be fully active.

- One to store list of peripheries that can put mcu to lower power mode (halt).

Every periphery, when it activates, adds object to certain linked list. After it finishes the process, that object is removed. For example, before UART communication object is added and when communication is done – removed. After scheduler runs through all active tasks, it cheks thouse linked lists. If they are both empty – puts device to deep sleep.

3. There is no CRA stuff mentioned in code because there was no time for that (just this part alone can take few days...). Now about those three points:
    a. For secure transport I would use AES encryption.
    b. Depends on mcu and what safety features does it have. But probably I would integrate key management system. Keys would be stored in protected memory behind firewall. There would be no way of retrieving value of those keys from application. It would only be possible to encrypt or decrypt data through predefined interface.
    c. Device encrypted firmware would be uploaded through lteM or other ways (NFC, optical interface etc) and put to external flash. Before update, new firmware would be verified and authenticated. If it succeeds, bootloader would read it from external flash, decrypt it, and store to mcu flash.
4. For troubleshooting I recommend using trice library. This library stores IDs of logs in code and creates local database with which logs can be reconstructed to full messages. That is way more memory efficient. That enables us to store

more logs to external flash or send it over air. With right tools field engineer can read them and provide feedback to the developers.

5. These are the thing that I would do if I would have more time:
    a. Finish modem control part
    b. Add external flash support
    c. Add bootloader and FOTA possibility
    d. Add device configuration module and possibility to change it over the air
    e. Add board management system